

PRUEBA TÉCNICA GUIADA

Spring Boot desde cero

Construye una API FinTech sin recibir el código resuelto

PROYECTO

Billetera Clara API

Requisitos, ruta de aprendizaje, investigación y evaluación

Preparado para Mikko Piaggio

Nivel: primer proyecto completo de Spring Boot | Modalidad: aprendizaje activo

Edición 2026

ANTES DE EMPEZAR

Cómo usar este cuaderno

Cómo aprovechar esta prueba

Este documento combina dos cosas: una **prueba técnica realista** y una **ruta de aprendizaje guiada**. Primero leerás el problema completo, como si una empresa te lo hubiera enviado. Después encontrarás una secuencia para construirlo desde cero aunque todavía no conozcas Spring Boot. La guía explica para qué sirve cada pieza, pero no muestra cómo programarla. La investigación y las decisiones siguen siendo tuyas.

Regla principal: este PDF no contiene la solución

No hay fragmentos de código, comandos listos para copiar, firmas de métodos, clases completas ni respuestas a las preguntas técnicas. Cuando aparezca un concepto nuevo, recibirás una explicación sencilla, una tarea de búsqueda y una evidencia que deberás producir.

Lo haré en verde pero luego lo traducire al ingles

Método de trabajo recomendado

- 1 Lee primero todo el enunciado sin abrir el IDE. **Subraya sustantivos que podrían convertirse en datos y verbos que podrían convertirse en operaciones.**
- 2 **Crea un documento llamado diario de decisiones. Cada vez que elijas una herramienta, tipo de dato o estructura, escribe por qué la elegiste y qué alternativa descartaste.**
- 3 Avanza una etapa por vez. No cruces una puerta de salida si no puedes explicar oralmente lo que acabas de construir.
- 4 Busca primero definiciones en documentación oficial. Después consulta tutoriales para ver ejemplos. Finalmente haz un experimento pequeño en tu propio proyecto.
- 5 **Cuando algo funcione, intenta explicar por qué funciona. Cuando falle, conserva el mensaje de error y anota qué hipótesis probaste.**
- 6 Al finalizar, presenta el proyecto como si estuvieras en una entrevista: problema, diseño, decisiones, pruebas y aspectos que mejorarías.

Qué significa 'sin código'

Sí encontrarás	No encontrarás
Qué componente debes crear y cuál es su responsabilidad.	La sintaxis exacta para crearlo.
Qué comportamiento debe observarse desde fuera.	Una implementación para copiar y pegar.
Conceptos explicados con lenguaje cotidiano.	La respuesta cerrada a cada decisión de diseño.
Preguntas, criterios de aceptación y pistas graduadas.	Una lista de anotaciones colocadas en el lugar exacto.
Formas de demostrar que tu solución funciona.	Un repositorio de referencia que elimine la investigación.

Tu objetivo real

El proyecto terminado importa, pero el objetivo principal es que puedas reconstruir mentalmente el recorrido completo de una petición: desde que llega por HTTP hasta que una regla de negocio modifica datos y la respuesta vuelve al cliente.

Índice

Cómo aprovechar esta prueba	2
Método de trabajo recomendado	2
Qué significa 'sin código'	2
1. Ficha del desafío	5
2. Contexto de negocio	5
3. Alcance obligatorio	5
4. Información que debe conservar el sistema	6
5. Reglas de negocio	6
6. Comportamiento de la API	6
7. Persistencia y consistencia	7
8. Validación y tratamiento de errores	7
9. Pruebas obligatorias	7
10. Entregables	8
11. Restricciones de la entrega	8
12. Escenarios de aceptación	9
13. Rúbrica de evaluación	9
14. Mejoras opcionales	10
15. El mapa mental completo	11
16. Glosario mínimo antes de tocar el IDE	11
Etapa 0. Preparar el entorno	12
Etapa 1. Generar el proyecto base	12
17. Anatomía del proyecto generado	14
Etapa 2. Arrancar y observar la aplicación vacía	14
Etapa 3. Diseñar paquetes y responsabilidades	15
18. Diseñar antes de programar	17
Plantilla para analizar un caso de uso	17
Etapa 4. Modelar el dominio	17
Etapa 5. Conectar la persistencia	18
Etapa 6. Definir entradas, salidas y validaciones	19
Etapa 7. Implementar los servicios y reglas	21
Etapa 8. Diseñar la capa web	22
Etapa 9. Unificar errores	22

Etapa 10. Proteger la transferencia con transacciones	24
Etapa 11. Construir una estrategia de pruebas	24
Etapa 12. Separar configuraciones	25
Etapa 13. Documentar y preparar la entrega	27
Etapa 14. Revisar como evaluador	27
19. Cómo completar una tarjeta	29
A. Java y construcción	29
B. Núcleo de Spring	30
C. Web y contratos	30
D. Persistencia	31
E. Dinero, tiempo y consistencia	31
F. Pruebas y operación	32
20. Plantilla para una tarjeta	33
21. Plan sugerido de 10 sesiones	34
22. Registro de sesión	34
23. Lista de control por hitos	35
24. Semáforo de comprensión	35
25. Guion de defensa de 15 minutos	36
26. Preguntas que podría hacer el evaluador	36
27. Hoja de autoevaluación	38
28. Lista final de requisitos	39
29. Declaración de autoría y comprensión	39
30. Última reflexión	39

PARTE I

La prueba técnica

1. Ficha del desafío

Aspecto	Definición
Nombre	Billetera Clara API
Nivel	Principiante en Spring Boot con base previa de Java y programación orientada a objetos
Duración sugerida	24 a 32 horas de trabajo, repartidas en 8 a 12 sesiones
Objetivo	Diseñar una API que administre clientes, billeteras y operaciones monetarias con historial auditable
Modalidad	Proyecto individual, repositorio Git y defensa oral de 15 a 20 minutos
Tecnologías base	Java, Spring Boot, Maven, API web, persistencia relacional y pruebas automatizadas
Restricción pedagógica	No comenzar con autenticación, interfaz gráfica, microservicios ni mensajería

2. Contexto de negocio

Una pequeña FinTech está preparando el **primer prototipo** de una billetera digital. Necesita un servicio de backend **capaz de registrar clientes, crear una billetera para cada uno, recibir depósitos, realizar retiros, transferir dinero y consultar el historial**. Todavía no habrá una aplicación móvil: otros programas se comunicarán con tu sistema mediante solicitudes web y recibirán datos estructurados como respuesta.

La empresa valora especialmente la claridad. Una operación monetaria no puede quedar a medias, un error debe explicarse de forma consistente y el historial debe permitir reconstruir por qué el saldo actual tiene determinado valor.

El papel que asumirás

Actuarás como desarrollador backend junior. No solo debes hacer que el sistema funcione: deberás organizarlo, justificar decisiones y demostrarlo con pruebas.

3. Alcance obligatorio

La primera versión debe cubrir seis capacidades completas:

Capacidad	Resultado observable
Registrar clientes	Guardar identidad básica y datos de contacto, rechazando duplicados e información inválida.
Crear billeteras	Asociar una billetera activa a un cliente existente, con saldo inicial igual a cero.
Depositar	Aumentar el saldo y registrar un movimiento exitoso con el saldo resultante.
Retirar	Disminuir el saldo solo cuando existe dinero suficiente y registrar el movimiento.
Transferir	Mover dinero entre dos billeteras distintas como una única operación indivisible.
Consultar	Obtener datos de clientes y billeteras, saldo actual e historial ordenado.

4. Información que debe conservar el sistema

Los siguientes campos describen el mínimo de negocio. **Tú deberás decidir qué parte se representa mediante entidades, objetos de entrada, objetos de salida, valores enumerados o relaciones.** También deberás justificar los tipos de datos elegidos.

Concepto	Información mínima
Cliente	Identificador, nombre completo, documento, correo electrónico, fecha y hora de registro y estado activo.
Billetera	Identificador, cliente propietario, moneda, saldo actual, estado y fecha y hora de creación.
Movimiento	Identificador, billetera, tipo, monto, fecha y hora, descripción, saldo resultante e identificador común de operación cuando corresponda.
Transferencia	Billetera de origen, billetera de destino, monto, referencia comprensible e identificador común para relacionar ambos movimientos.

Decisión deliberadamente abierta

El enunciado no define si una transferencia merece una entidad propia ni qué estrategia de identificadores debes usar. Investiga ambas opciones, elige una y documenta sus consecuencias.

5. Reglas de negocio

- [] El documento y el correo electrónico identifican datos únicos de cada cliente.
- [] Un cliente inactivo no puede recibir una nueva billetera ni operar una existente.
- [] En la versión obligatoria, cada cliente puede tener como máximo una billetera activa.
- [] El saldo inicial de una billetera es cero y no puede establecerlo libremente quien crea la billetera.
- [] La moneda obligatoria es ARS. El diseño debe evitar que una operación futura mezcle monedas sin control.
- [] Todo monto monetario debe ser mayor que cero y admitir como máximo dos cifras decimales.
- [] Un retiro no puede dejar saldo negativo.
- [] Una transferencia no puede tener la misma billetera como origen y destino.
- [] Una transferencia solo se completa si ambas billeteras existen, están activas y el origen tiene fondos suficientes.
- [] Una transferencia exitosa produce dos movimientos: uno enviado y otro recibido, unidos por un identificador común.
- [] El depósito, retiro o transferencia rechazados no generan movimientos exitosos.
- [] La fecha y la hora de una operación las decide el servidor, no el cliente externo.
- [] Un movimiento representa un hecho histórico y no se modifica ni elimina mediante la API ordinaria.
- [] El historial se devuelve del movimiento más reciente al más antiguo.
- [] El saldo resultante guardado en cada movimiento debe coincidir con el saldo inmediatamente posterior a esa operación.

6. Comportamiento de la API

Debes diseñar recursos y operaciones web coherentes. El evaluador no impone rutas exactas: se medirá si los nombres son claros, si el método HTTP concuerda con la intención, si el estado de respuesta representa el resultado y si el cuerpo contiene solamente información útil.

- [] Definir operaciones para crear y consultar clientes.
- [] Definir operaciones para crear y consultar billeteras.
- [] Definir operaciones separadas para depósito, retiro y transferencia.
- [] Permitir consultar el historial de una billetera.

- [] Permitir filtrar el historial por tipo y por intervalo de fechas.
- [] Pagar el historial o justificar por qué se dejó como mejora futura.
- [] Usar un formato uniforme para todos los errores.
- [] Distinguir entre datos inválidos, recurso inexistente, conflicto de negocio y error inesperado.
- [] No exponer detalles internos de la base de datos ni trazas técnicas en las respuestas públicas.

7. Persistencia y consistencia

La información debe sobrevivir al reinicio normal de la aplicación cuando se utilice la configuración de desarrollo persistente. Durante las primeras etapas puedes trabajar con una base en memoria, pero la entrega deberá incluir una opción de base de datos relacional persistente.

- Modelar correctamente las relaciones entre cliente, billetera y movimientos.
- Definir restricciones tanto en la aplicación como en la base de datos cuando corresponda.
- Evitar usar números binarios de punto flotante para representar dinero.
- Asegurar que la transferencia completa se confirme o se revierta como una unidad.
- Analizar qué ocurriría si dos retiros simultáneos intentaran gastar el mismo saldo.
- Separar la configuración local de la configuración de pruebas.

8. Validación y tratamiento de errores

Validar significa impedir que datos incorrectos lleguen al corazón del sistema. Manejar errores significa convertir un problema técnico o de negocio en una respuesta estable que otra persona pueda comprender y procesar. Ambas responsabilidades deben estar separadas de la lógica feliz.

Situación	Comportamiento esperado
Nombre vacío o correo con formato inválido	Rechazo antes de intentar guardar y detalle del campo afectado.
Cliente, billetera o movimiento inexistente	Respuesta que indique que el recurso solicitado no existe.
Documento o correo duplicado	Respuesta de conflicto sin revelar información interna.
Monto igual a cero, negativo o con escala inválida	Rechazo de la solicitud con explicación del dato inválido.
Fondos insuficientes o billetera inactiva	Rechazo como regla de negocio; ningún cambio parcial debe persistir.
Error inesperado	Respuesta genérica y registro técnico suficiente para diagnosticarlo.

9. Pruebas obligatorias

No se evaluará una colección enorme de pruebas repetidas. Se valoran escenarios seleccionados que demuestren reglas críticas y límites entre capas. Debes poder explicar qué unidad protege cada prueba y por qué fallaría si aparece una regresión.

- [] Pruebas unitarias del servicio para depósito, retiro y transferencia.
- [] Prueba del rechazo de un retiro por fondos insuficientes.
- [] Prueba de que una transferencia produce los dos movimientos correctos.
- [] Prueba de que una transferencia fallida no deja cambios parciales.
- [] Pruebas de validación de entradas inválidas.
- [] Prueba de integración de al menos un recorrido completo desde la capa web hasta la base de pruebas.
- [] Prueba del formato de error para un recurso inexistente.
- [] Prueba del orden del historial.

10. Entregables

- [] Repositorio Git con historial de cambios comprensible.
- [] Proyecto ejecutable mediante la herramienta de construcción elegida.
- [] Archivo de instrucciones con propósito, requisitos previos, pasos de ejecución, decisiones y limitaciones.
- [] Descripción de la API mediante una especificación navegable o una colección de solicitudes reproducibles.
- [] Configuración de desarrollo y de pruebas sin credenciales reales.
- [] Pruebas automatizadas ejecutables y reporte final de resultados.
- [] Diagrama sencillo del modelo de datos y otro del recorrido de una petición.
- [] Diario de decisiones con al menos cinco decisiones justificadas.

11. Restricciones de la entrega

- No crear una interfaz gráfica: la API es el producto evaluado.
- No dividir el sistema en microservicios.
- No añadir autenticación hasta completar todos los requisitos obligatorios.
- No guardar credenciales, secretos ni archivos generados por la construcción en Git.
- No devolver directamente objetos de persistencia como contrato público sin justificarlo.
- No concentrar reglas de negocio en controladores ni en clases de acceso a datos.
- No usar una librería para ocultar conceptos que todavía no puedes explicar.

Sobre las ayudas automáticas

Puedes usar documentación, buscadores y asistentes para aclarar conceptos. No debes aceptar una solución que no puedas explicar línea por línea. En la defensa se evaluará tu modelo mental, no la velocidad con la que generaste archivos.

12. Escenarios de aceptación

Estos escenarios describen comportamientos externos. No dicen cómo implementarlos. Conviértelos en pruebas manuales y, cuando corresponda, automatizadas.

ID	Escenario	Resultado observable
A01	Alta válida	Al registrar un cliente nuevo con datos correctos, el sistema confirma la creación y permite consultarlo.
A02	Duplicado	Al registrar otro cliente con el mismo documento o correo, el sistema rechaza la operación sin crear un segundo registro.
A03	Billetera inicial	Al crear la billetera de un cliente activo, el saldo visible es exactamente cero y la moneda es ARS.
A04	Segundo alta	Al intentar crear otra billetera activa para el mismo cliente, el sistema rechaza la operación.
A05	Depósito	Al depositar un monto válido, el saldo aumenta una sola vez y aparece un movimiento con el saldo resultante.
A06	Monto inválido	Al depositar cero o un valor negativo, no cambia el saldo y no aparece un movimiento.
A07	Retiro válido	Al retirar un monto menor o igual al saldo, el nuevo saldo y el movimiento son coherentes.
A08	Sin fondos	Al retirar más dinero del disponible, la operación se rechaza y todos los datos quedan intactos.
A09	Transferencia	Al transferir entre billeteras activas, el origen disminuye, el destino aumenta y aparecen dos movimientos relacionados.
A10	Mismo destino	Al usar la misma billetera como origen y destino, la transferencia se rechaza.
A11	Fallo intermedio	Si la transferencia no puede completarse, ninguna de sus partes queda confirmada.
A12	Historial	Al consultar movimientos, el más reciente aparece primero y los filtros devuelven únicamente coincidencias.
A13	No encontrado	Al consultar un identificador inexistente, la respuesta conserva el formato estándar de error.
A14	Reinicio	Al reiniciar con el perfil persistente, los clientes, saldos y movimientos siguen disponibles.

13. Rúbrica de evaluación

Área	Puntos	Qué se observa
Comprensión del dominio	10	Reglas claras, nombres comprensibles y ausencia de contradicciones.
Modelo y persistencia	15	Relaciones, restricciones, tipos monetarios y esquema coherentes.
Lógica de negocio	20	Operaciones correctas, historial confiable y casos rechazados sin efectos.
Diseño de API	15	Recursos, métodos, estados, entradas y salidas consistentes.
Transacciones y consistencia	10	Transferencias atómicas y análisis explícito de concurrencia.
Validación y errores	10	Datos inválidos y conflictos convertidos en respuestas uniformes.
Pruebas	12	Escenarios críticos, aislamiento correcto y al menos un recorrido integrado.
Calidad y documentación	8	Estructura legible, configuración segura, Git y guía de uso.

Criterio para considerarla aprobada

Una entrega alcanza el nivel esperado con 70 puntos o más, siempre que la transferencia sea atómica, el dinero no use punto flotante binario y las reglas críticas tengan pruebas.

14. Mejoras opcionales

Solo aborda estas extensiones después de cumplir y verificar todo lo obligatorio.

- Migraciones versionadas de base de datos.
- Documentación interactiva de la API.
- Contenedores para la aplicación y la base de datos.
- Idempotencia para evitar duplicar una operación repetida por problemas de red.
- Bloqueo o control optimista para operaciones monetarias concurrentes.
- Autenticación y autorización por propietario de billetera.
- Métricas básicas y comprobación de salud de la aplicación.
- Conversión del historial a páginas con metadatos de navegación.

PARTE II

Construcción desde cero, sin código

15. El mapa mental completo

Antes de crear archivos, imagina el sistema como una oficina. Una solicitud llega a recepción, alguien traduce el pedido, otra persona aplica las reglas, un archivista lee o guarda información y finalmente la oficina devuelve una respuesta. Spring Boot ayuda a conectar esas responsabilidades y a iniciar la oficina con una configuración predecible.

Recepción	Traducción	Reglas	Archivado	Respuesta
Envía una solicitud	Traduce HTTP	Aplica reglas	Accede a datos	Conserva el estado

Las flechas no son una regla matemática: son una disciplina para saber quién puede pedirle trabajo a quién. La capa web conoce el lenguaje de internet; el servicio conoce el negocio; el repositorio conoce la forma de recuperar y guardar; la base conserva datos. Tus decisiones deben mantener esas fronteras.

16. Glosario mínimo antes de tocar el IDE

Término	Explicación inicial, todavía incompleta
Java	El lenguaje en el que expresarás las clases, reglas y pruebas.
JVM	El entorno que ejecuta las instrucciones compiladas de Java.
JDK	El conjunto de herramientas necesario para desarrollar, compilar y ejecutar Java.
Spring	Un ecosistema que administra objetos de la aplicación y ofrece módulos para problemas habituales.
Spring Boot	Una forma de iniciar aplicaciones Spring con convenciones, configuración automática y servidor integrado.
Maven	Una herramienta que describe el proyecto, obtiene dependencias y ejecuta fases de construcción.
Dependencia	Una biblioteca externa que el proyecto necesita para cumplir una responsabilidad.
Bean	Un objeto cuya creación y ciclo de vida administra el contenedor de Spring.
Inyección de dependencias	Una forma de recibir colaboraciones desde afuera en vez de construirlas dentro de cada clase.
API	Un contrato que indica cómo otros programas pueden pedir operaciones o datos.
HTTP	El protocolo usado para enviar solicitudes y respuestas por la web.
JSON	Una representación textual estructurada habitual para intercambiar datos.
Persistencia	La capacidad de conservar información más allá de un objeto temporal en memoria.
ORM	Un mecanismo que relaciona objetos del programa con filas y tablas.
Transacción	Un límite que hace que varias modificaciones se confirmen juntas o se reviertan juntas.

Etapa 0. Preparar el entorno

El entorno es el taller donde construirás. El JDK aporta las herramientas de Java, el IDE te ayuda a navegar y detectar errores, Maven organiza dependencias y tareas, Git registra la historia, el cliente HTTP simula a quien usará la API y la base de datos conserva información.

Lo que debes crear

- [] Instalar o verificar una versión LTS del JDK compatible con la versión de Spring Boot elegida.
- [] Configurar IntelliJ IDEA para usar ese JDK como SDK del proyecto.
- [] Verificar que Maven pueda utilizarse mediante el wrapper que generará el proyecto.
- [] Preparar Git y un repositorio vacío.
- [] Instalar un cliente para enviar solicitudes HTTP.
- [] Preparar PostgreSQL para la entrega; reservar una base en memoria para las primeras prácticas y las pruebas.

Antes de hacerlo, investiga

- Diferencia entre JDK, JRE y JVM.
- Qué significa que una versión de Java sea LTS.
- Qué problema resuelve Maven y qué es un wrapper.
- Qué información muestra la variable de entorno que localiza el JDK.
- Diferencia entre cliente HTTP, servidor web y base de datos.

Evidencias que debes guardar

- [] Captura o anotación con la versión del JDK detectada por el IDE.
- [] Una frase propia para cada herramienta y su responsabilidad.
- [] Primer commit que contenga solamente la estructura inicial del repositorio.

Preguntas de control

- 1 ¿Por qué instalar solo un entorno de ejecución no alcanza para desarrollar?
- 2 ¿Qué tarea realiza Maven que no debería realizar manualmente el IDE?
- 3 ¿Por qué el cliente HTTP no forma parte de la aplicación?
- 4 ¿Qué datos no deberían subirse nunca al repositorio?

Errores comunes que debes vigilar

Confundir el JDK del sistema con el seleccionado por el IDE; depender de una instalación global de Maven; guardar contraseñas en archivos versionados; intentar instalar muchas herramientas opcionales antes de ejecutar el proyecto mínimo.

Puerta de salida

Puedes señalar cada herramienta y explicar, sin leer, qué problema resuelve. El IDE reconoce el JDK correcto y el repositorio está limpio y listo.

Etapa 1. Generar el proyecto base

Spring Initializr es un formulario que fabrica la estructura inicial. No crea tu aplicación de negocio: prepara metadatos, carpetas, configuración de construcción y dependencias seleccionadas para que comiences desde un punto coherente.

Lo que debes crear

- [] Generar un proyecto Maven con Java y empaquetado ejecutable.
- [] Elegir una versión estable de Spring Boot y una versión LTS compatible de Java.
- [] Definir un grupo, nombre de artefacto, nombre visible, descripción y paquete base propios.
- [] Seleccionar capacidades para web, validación, persistencia relacional, base en memoria, controlador de PostgreSQL y pruebas.
- [] Abrir el proyecto como proyecto Maven, esperar la descarga de dependencias y ejecutar la prueba inicial.

Antes de hacerlo, investiga

- Qué significan grupo, artefacto, paquete, empaquetado y versión.
- Qué diferencia existe entre una dependencia inicial y una clase que tú escribirás.
- Para qué sirven las capacidades de web, validación y acceso a datos.
- Por qué una dependencia de base en memoria y otra de PostgreSQL pueden convivir.
- Qué son una versión estable, una versión candidata y una instantánea de desarrollo.

Evidencias que debes guardar

- [] Captura o nota con las opciones elegidas y una justificación breve.
- [] Lista de dependencias seleccionadas explicada con palabras propias.
- [] Registro de una construcción inicial exitosa antes de agregar negocio.

Preguntas de control

- 1 ¿Quién descargará las bibliotecas que seleccionaste?
- 2 ¿Qué diferencia conceptual hay entre Spring y Spring Boot?
- 3 ¿Qué consecuencias tiene elegir un nombre de paquete que no controlas?
- 4 ¿Por qué conviene comenzar con pocas dependencias?

Errores comunes que debes vigilar

Elegir una versión experimental sin saberlo; agregar todas las dependencias disponibles; abrir una carpeta interna en vez de la raíz; modificar archivos mientras Maven todavía está importando; usar una versión de Java incompatible.

Puerta de salida

El proyecto vacío construye y sus pruebas iniciales terminan correctamente. Puedes justificar cada opción seleccionada sin responder 'porque venía marcada'.

17. Anatomía del proyecto generado

Antes de crear nuevas clases, recorre lo que ya existe. Estos elementos parecen detalles, pero entenderlos evita que Spring Boot se sienta como magia. No memorices el contenido completo: aprende quién lo usa, cuándo cambia y qué error aparece si está mal configurado.

Elemento	Para qué existe	Qué debes investigar
Descriptor de Maven	Declara identidad, versión, dependencias y tareas de construcción.	Cómo se organizan sus secciones; qué aporta el padre de Spring Boot; diferencia entre dependencia y complemento.
Wrapper de Maven	Permite usar una versión prevista de la herramienta sin depender de una instalación global.	Qué archivos lo componen y por qué deben versionarse.
Carpeta principal de Java	Contiene el código que forma la aplicación.	Cómo el paquete base condiciona el descubrimiento de componentes.
Clase de arranque	Contiene el punto de entrada que inicia el contexto de Spring.	Qué ocurre entre el inicio de Java y el mensaje de aplicación arrancada.
Recursos	Guarda configuración y otros archivos que deben estar disponibles al ejecutar.	Diferencia entre propiedades generales, perfiles y recursos estáticos.
Carpeta de pruebas	Contiene verificaciones que no forman parte del producto ejecutado.	Cómo se relaciona su paquete con las clases probadas.
Directorio de salida	Recibe artefactos generados, clases compiladas e informes.	Por qué se regenera y no debe editarse ni subirse a Git.

Ejercicio de reconstrucción

Cierra el IDE y dibuja de memoria el árbol mínimo del proyecto. Al lado de cada elemento escribe: quién lo lee, cuándo se usa y si pertenece a tu código fuente o es un resultado generado.

Etapas 2. Arrancar y observar la aplicación vacía

Una aplicación de Spring Boot comienza como un programa Java normal y luego construye un contexto: un espacio administrado donde registra objetos, configura módulos y arranca un servidor integrado. Los mensajes de inicio cuentan esa historia si aprendes a leerlos.

Lo que debes crear

- [] Ejecutar la clase de arranque desde el IDE.
- [] Identificar en los registros el puerto, el tiempo de inicio, el perfil activo y el servidor web.
- [] Detener la aplicación de forma controlada y volver a iniciarla.
- [] Cambiar únicamente el puerto mediante configuración y comprobar el efecto.
- [] Crear una nota de diagnóstico para el caso de puerto ocupado.

Antes de hacerlo, investiga

- Qué es el método de entrada de Java y qué devuelve el arranque de Spring.
- Qué es el contexto de aplicación.
- Qué significa servidor web integrado.
- Qué es un perfil de configuración.
- Cómo distinguir advertencias de errores en los registros.

Evidencias que debes guardar

- [] Fragmento escrito con los cuatro mensajes de inicio que consideres más importantes, sin copiar información sensible.
- [] Explicación de por qué todavía recibirías un resultado de recurso inexistente al visitar el servidor.
- [] Una entrada en el diario sobre el cambio de puerto.

Preguntas de control

- 1 ¿Qué parte pertenece a Java y qué parte agrega Spring Boot?
- 2 ¿Por qué puede existir un servidor aunque todavía no hayas creado una ruta?
- 3 ¿Qué diferencia hay entre que la aplicación compile y que arranque correctamente?
- 4 ¿Qué revisarías primero si el puerto ya está ocupado?

Errores comunes que debes vigilar

Interpretar toda línea roja del IDE como un error de ejecución; mantener dos instancias usando el mismo puerto; cambiar varias configuraciones al mismo tiempo; ignorar el primer error real y leer solamente la última consecuencia.

Puerta de salida

Puedes iniciar y detener la aplicación, localizar la información esencial en los registros y explicar por qué una aplicación vacía ya funciona como servidor.

Etapas 3. Diseñar paquetes y responsabilidades

Una capa no es simplemente una carpeta. Es una frontera de responsabilidad. Separar el transporte web, las reglas de negocio y la persistencia hace que una modificación tenga un lugar natural y que una prueba pueda aislar aquello que quiere comprobar.

Lo que debes crear

- [] Crear paquetes para dominio, acceso a datos, servicios, capa web, objetos de intercambio, manejo de errores y configuración.
- [] Escribir una breve regla de dependencia que indique qué capas pueden conocer a cuáles.
- [] Crear un diagrama del recorrido de una solicitud desde el cliente hasta la base y de regreso.
- [] Decidir si organizarás primero por tipo técnico o por característica de negocio, y justificarlo.

Antes de hacerlo, investiga

- Qué significan arquitectura por capas, cohesión y acoplamiento.
- Qué responsabilidad distingue a un controlador, un servicio y un repositorio.
- Qué es una dependencia circular y por qué dificulta el diseño.
- Diferencia entre organizar paquetes por capa y por funcionalidad.
- Por qué la clase de arranque suele ubicarse por encima de los demás paquetes.

Evidencias que debes guardar

- [] Diagrama de capas con una frase de responsabilidad por capa.
- [] Registro de la estrategia de paquetes elegida y una alternativa descartada.
- [] Lista de tres cosas que nunca deberían resolverse en el controlador.

Preguntas de control

- 1 ¿Dónde viviría la regla de fondos suficientes y por qué?
- 2 ¿Quién debería saber que la comunicación usa HTTP?

- 3 ¿Quién debería saber cómo consultar la base de datos?
- 4 ¿Qué síntoma indica que una clase tiene demasiadas responsabilidades?

Errores comunes que debes vigilar

Crear carpetas sin mantener límites reales; poner toda la lógica en el controlador porque es el primer lugar que recibe la petición; permitir que una entidad construya servicios; usar un paquete genérico que termina conteniendo todo.

Puerta de salida

Cada componente planeado tiene una sola responsabilidad principal y puedes narrar el recorrido de una petición sin mezclar HTTP con reglas monetarias.

18. Diseñar antes de programar

El diseño inicial no pretende acertar para siempre. Sirve para hacer visibles las decisiones. Usa lápiz, papel o una herramienta de diagramas y crea tres mapas: datos, operaciones y errores. Luego compáralos con el enunciado para detectar huecos.

Mapa	Pregunta central	Resultado mínimo
Datos	¿Qué información existe y cómo se relaciona?	Cliente, billetera, movimiento, identificadores, cardinalidades y restricciones.
Operaciones	¿Qué puede pedir un cliente externo?	Entrada, precondiciones, pasos de negocio, salida y efectos para cada caso de uso.
Errores	¿Qué puede impedir cada operación?	Catálogo con categoría, mensaje público, estado HTTP esperado y ausencia o presencia de efectos.

Plantilla para analizar un caso de uso

Pregunta	Tu respuesta
¿Quién solicita la operación?	
¿Qué datos entrega?	
¿Qué validaciones de forma deben cumplirse?	
¿Qué recursos deben existir?	
¿Qué reglas de negocio se evalúan?	
¿Qué datos cambian si todo sale bien?	
¿Qué debe ocurrir si algo falla a mitad?	
¿Qué respuesta externa confirma el resultado?	
¿Qué pruebas demostrarán el comportamiento?	

Etapas 4. Modelar el dominio

El dominio es el lenguaje del problema. Una entidad representa algo que conserva identidad a través del tiempo; un valor describe una característica; una regla protege estados válidos. Aquí decides cómo expresar clientes, billeteras, movimientos y tipos de operación sin pensar todavía en rutas web.

Lo que debes crear

- ☐ Crear las entidades necesarias para cliente, billetera y movimiento.
- ☐ Crear una representación limitada para moneda, estado de billetera y tipo de movimiento.
- ☐ Definir identificadores y relaciones con cardinalidades explícitas.
- ☐ Elegir tipos apropiados para dinero, fecha y hora.
- ☐ Proteger el saldo y el historial contra modificaciones arbitrarias desde otras clases.
- ☐ Decidir dónde se crea el identificador común de una transferencia.

Antes de hacerlo, investiga

- Diferencia entre entidad, objeto de valor y objeto de transferencia.
- Por qué el dinero requiere precisión decimal exacta.
- Diferencia entre instante global y fecha y hora sin zona.
- Ventajas y costos de identificadores numéricos y universales.

- Qué significan cardinalidad, propiedad de la relación y carga diferida.
- Qué implica que un movimiento sea inmutable desde el negocio.

Evidencias que debes guardar

- [] Diagrama entidad-relación con cardinalidades.
- [] Diccionario de datos con tipo, obligatoriedad y regla por campo.
- [] Decisión escrita sobre dinero, tiempo e identificadores.
- [] Ejemplo en lenguaje natural de un estado válido y uno inválido para cada entidad.

Preguntas de control

- 1 ¿Por qué el saldo no debería ser un número de punto flotante binario?
- 2 ¿Qué diferencia existe entre identificar un movimiento y agrupar los dos lados de una transferencia?
- 3 ¿Quién puede modificar el saldo y bajo qué condiciones?
- 4 ¿Necesitas almacenar una lista de movimientos dentro de la billetera para poder consultarlos?
- 5 ¿Qué zona horaria usarás internamente y cómo la explicarás?

Errores comunes que debes vigilar

Diseñar entidades como simples bolsas públicas de datos; permitir que cualquier clase cambie el saldo; usar texto libre para categorías cerradas; crear relaciones bidireccionales sin necesidad; confundir la escala monetaria con el redondeo.

Puerta de salida

El modelo puede representar todos los escenarios obligatorios, impide estados evidentemente inválidos y cada tipo importante está justificado en el diario.

Etapas 5. Conectar la persistencia

Los objetos viven en memoria y desaparecen al terminar el proceso. La persistencia traduce su estado a una base relacional. JPA define reglas comunes para esa traducción; el proveedor realiza el trabajo concreto; Spring Data ayuda a declarar accesos habituales sin escribir toda la infraestructura.

Lo que debes crear

- [] Convertir las clases persistentes en entidades reconocibles por el proveedor.
- [] Definir claves, columnas, restricciones y relaciones coherentes con el diagrama.
- [] Crear un repositorio por raíz de datos que realmente necesites consultar.
- [] Añadir operaciones de búsqueda por documento, correo, propietario e historial ordenado.
- [] Configurar una base en memoria para aprendizaje y pruebas.
- [] Configurar PostgreSQL mediante valores externos para desarrollo persistente.

Antes de hacerlo, investiga

- Diferencia entre JPA, un proveedor ORM, Spring Data y una base de datos.
- Qué condiciones debe cumplir una entidad persistente.
- Cómo se convierten nombres de clases y propiedades en tablas y columnas.
- Qué aportan las restricciones únicas y las claves foráneas.
- Cómo deriva consultas Spring Data a partir de nombres y cuándo deja de ser conveniente.
- Qué significan carga ansiosa, carga diferida y problema de múltiples consultas repetidas.

Evidencias que debes guardar

- [] Esquema observado en la base y comparación con tu diagrama.
- [] Prueba manual de guardar y recuperar un registro.
- [] Lista de restricciones protegidas por la base y por la aplicación.
- [] Registro de configuración donde ningún secreto real aparezca en Git.

Preguntas de control

- 1 ¿Qué trabajo hace el repositorio y cuál no debería hacer?
- 2 ¿Por qué validar unicidad solo con una consulta previa puede ser insuficiente?
- 3 ¿Qué sucede con los datos de una base en memoria al detener la aplicación?
- 4 ¿Qué problema aparece si cargas siempre todas las relaciones?
- 5 ¿Por qué generar automáticamente el esquema puede ser útil al aprender y riesgoso en producción?

Errores comunes que debes vigilar

Confundir el proveedor ORM con la especificación; almacenar secretos en la configuración versionada; usar generación destructiva del esquema sin comprenderla; devolver colecciones enormes; resolver reglas monetarias dentro del repositorio.

Puerta de salida

Puedes guardar, reiniciar según el perfil elegido y recuperar datos. El esquema respeta el diseño y puedes explicar el recorrido objeto-tabla-objeto.

Etapas 6. Definir entradas, salidas y validaciones

La forma en que guardas datos no tiene por qué ser la forma en que hablas con el exterior. Los objetos de entrada expresan qué acepta una operación; los de salida expresan qué promete la API. Esta separación protege el contrato y evita que un cambio interno se filtre accidentalmente.

Lo que debes crear

- [] Crear un objeto de entrada específico para registrar clientes.
- [] Crear objetos de entrada distintos para depósito, retiro y transferencia.
- [] Crear salidas para cliente, billetera, movimiento, operación exitosa y error.
- [] Definir validaciones de presencia, formato, rango y cantidad de decimales.
- [] Crear un mecanismo de conversión entre objetos externos y objetos del dominio.
- [] Decidir qué campos nunca aceptará el cliente porque pertenecen al servidor.

Antes de hacerlo, investiga

- Qué significa DTO y por qué no es simplemente una entidad duplicada.
- Diferencia entre validación sintáctica y regla de negocio.
- Cómo funciona la validación declarativa de objetos.
- Qué es serialización y cómo los nombres y tipos llegan a JSON.
- Ventajas y costos del mapeo manual y del mapeo automatizado.

Evidencias que debes guardar

- [] Tabla que relacione cada operación con su entrada y salida.
- [] Lista de campos controlados por el servidor.
- [] Cinco solicitudes inválidas y el error público esperado.
- [] Decisión sobre estrategia de mapeo.

Preguntas de control

- 1 ¿Por qué el cliente no debería enviar el saldo resultante?
- 2 ¿Validar fondos suficientes pertenece al formato de entrada o al negocio?
- 3 ¿Qué daño puede causar exponer directamente una entidad?
- 4 ¿Una salida debe incluir toda la información disponible?
- 5 ¿Cómo representarás varios errores de campo en una sola respuesta?

Errores comunes que debes vigilar

Reutilizar un único objeto para todas las operaciones; aceptar identificadores o fechas que debería producir el servidor; confundir campo ausente con recurso inexistente; dejar que mensajes internos se conviertan en contrato público.

Puerta de salida

Cada operación tiene un contrato explícito, los datos externos se validan antes del negocio y ninguna salida expone accidentalmente detalles internos.

Etapla 7. Implementar los servicios y reglas

El servicio coordina un caso de uso. Busca lo necesario, verifica reglas, ordena cambios y construye el resultado. Piensa en él como el director de una escena: no almacena físicamente los datos ni habla HTTP, pero decide qué colaboradores actúan y en qué orden.

Lo que debes crear

- [] Crear servicios para clientes y billeteras con dependencias recibidas desde el exterior.
- [] Implementar alta y consulta de clientes.
- [] Implementar creación de billetera.
- [] Implementar depósito y retiro.
- [] Implementar transferencia con dos movimientos relacionados.
- [] Implementar consulta de saldo e historial filtrable.
- [] Crear excepciones del dominio para resultados rechazados previsibles.

Antes de hacerlo, investiga

- Qué diferencia existe entre coordinación y lógica de infraestructura.
- Por qué se prefiere recibir dependencias mediante el constructor.
- Qué es inversión de control y cómo se relaciona con un bean.
- Diferencia entre excepción de negocio y fallo técnico inesperado.
- Qué significa que una operación sea atómica.
- Cómo decidir el orden de verificaciones sin producir efectos parciales.

Evidencias que debes guardar

- [] Secuencia escrita de pasos para cada operación monetaria.
- [] Tabla de precondiciones y efectos.
- [] Pruebas unitarias iniciales de los caminos exitosos y rechazados.
- [] Explicación de por qué el servicio no conoce estados HTTP.

Preguntas de control

- 1 ¿En qué momento exacto se crea un movimiento?
- 2 ¿Qué se debe comprobar antes de modificar un saldo?
- 3 ¿Por qué una transferencia no equivale simplemente a llamar retiro y luego depósito sin más análisis?
- 4 ¿Qué responsabilidad tendría una clase demasiado grande y cómo la dividirías?
- 5 ¿Cómo demostrarías que una operación rechazada no cambió nada?

Errores comunes que debes vigilar

Guardar antes de terminar todas las validaciones; capturar cualquier error y continuar; mezclar estados HTTP con reglas de negocio; construir manualmente dependencias dentro del servicio; duplicar reglas en varios métodos.

Puerta de salida

Todos los casos de uso funcionan desde pruebas del servicio, los rechazos no generan efectos y puedes describir cada secuencia sin mencionar detalles web.

Etapa 8. Diseñar la capa web

El controlador es un adaptador. Recibe una solicitud HTTP, pide que se valide y traduce la intención al servicio. Luego convierte el resultado en una respuesta. No decide si hay fondos suficientes: esa regla debe funcionar incluso si mañana la operación llega por otra interfaz.

Lo que debes crear

- [] Crear controladores separados por recurso o responsabilidad coherente.
- [] Diseñar rutas basadas en recursos, no en nombres internos de métodos.
- [] Asignar un método HTTP y un estado de respuesta adecuado a cada operación.
- [] Conectar entradas validadas con servicios y salidas públicas.
- [] Agregar consultas por identificador y filtros de historial.
- [] Probar manualmente cada escenario mediante el cliente HTTP.

Antes de hacerlo, investiga

- Estructura de una solicitud y una respuesta HTTP.
- Semántica de los métodos HTTP más usados.
- Diferencia entre ruta, parámetro de ruta, parámetro de consulta, cabecera y cuerpo.
- Familias de estados HTTP y criterios para creación, éxito, entrada inválida, ausencia y conflicto.
- Qué significa que una API esté orientada a recursos.
- Qué es negociación y serialización de contenido.

Evidencias que debes guardar

- [] Catálogo de operaciones con método, ruta propuesta, entrada, salida y estado.
- [] Colección de solicitudes manuales con casos felices y fallidos.
- [] Capturas o resultados guardados de al menos los catorce escenarios de aceptación.
- [] Justificación de dos decisiones de diseño de rutas.

Preguntas de control

- 1 ¿Qué diferencia hay entre una entrada inválida y un recurso inexistente?
- 2 ¿Qué operación puede repetirse sin cambiar el resultado y cuál podría duplicar dinero?
- 3 ¿Cuándo un filtro pertenece a la ruta y cuándo a la consulta?
- 4 ¿Por qué devolver éxito para todos los resultados dificulta a los clientes?
- 5 ¿Qué lógica permanecería si reemplazaras HTTP por una interfaz de consola?

Errores comunes que debes vigilar

Crear rutas como verbos arbitrarios sin modelo de recursos; usar siempre el mismo estado; escribir reglas en el controlador; aceptar entidades directamente; probar solo con el navegador; ignorar tipos de contenido.

Puerta de salida

Un cliente externo puede recorrer los casos de aceptación y cada respuesta comunica claramente éxito o causa de rechazo sin filtrar la implementación interna.

Etapa 9. Unificar errores

Los errores también forman parte del contrato. Un manejador central evita repetir conversiones en cada controlador y permite que todos los fallos conocidos tengan la misma estructura. El registro técnico puede ser detallado; el mensaje público debe ser seguro y útil.

Lo que debes crear

- [] Crear un modelo público de error con identificador temporal, estado, categoría, mensaje, ruta y detalles de campos cuando existan.
- [] Crear un manejador central para validación, recurso inexistente, conflicto y reglas monetarias.
- [] Definir un comportamiento seguro para errores inesperados.
- [] Asociar cada excepción de negocio con una respuesta coherente.
- [] Agregar un identificador de correlación opcional para diagnóstico.

Antes de hacerlo, investiga

- Diferencia entre excepción comprobada, no comprobada y error de dominio.
- Cómo funciona un manejador global en la capa web.
- Qué datos ayudan al cliente y cuáles exponen la implementación.
- Diferencia entre mensaje de registro y mensaje de respuesta.
- Qué significa correlacionar una solicitud con sus registros.

Evidencias que debes guardar

- [] Catálogo de errores con ejemplo en lenguaje natural.
- [] Pruebas de formato para al menos cuatro categorías.
- [] Comprobación de que una traza técnica no aparece en la respuesta pública.

Preguntas de control

- 1 ¿Por qué no conviene capturar toda excepción dentro de cada controlador?
- 2 ¿Qué información necesita otro programa para reaccionar sin leer texto humano?
- 3 ¿Una violación de unicidad es siempre un error inesperado?
- 4 ¿Qué parte del error puede cambiar sin romper clientes?

Errores comunes que debes vigilar

Devolver mensajes distintos para el mismo problema; exponer nombres de tablas o consultas; usar excepciones como flujo normal innecesario; ocultar el error técnico sin dejar registro; responder éxito con un mensaje de error dentro.

Puerta de salida

Todos los rechazos conocidos comparten una estructura estable, los estados son coherentes y existe información diagnóstica sin exposición de secretos.

Etapla 10. Proteger la transferencia con transacciones

Una transferencia modifica varias piezas relacionadas. Si una se guarda y otra falla, el sistema queda mintiendo. Una transacción establece una frontera: todo el conjunto se confirma si termina bien o vuelve al estado anterior si no puede completarse.

Lo que debes crear

- [] Delimitar la transferencia como una única unidad transaccional.
- [] Incluir dentro del mismo límite los dos saldos y los dos movimientos.
- [] Crear una prueba que provoque un fallo después del primer cambio y verifique la reversión.
- [] Analizar el caso de dos operaciones simultáneas sobre la misma billetera.
- [] Elegir una primera estrategia de control de concurrencia y documentar su costo.

Antes de hacerlo, investiga

- Propiedades de atomicidad, consistencia, aislamiento y durabilidad.
- Qué condiciones provocan confirmación o reversión.
- Dónde conviene colocar un límite transaccional.
- Qué es una actualización perdida.
- Diferencia entre bloqueo optimista y pesimista.
- Cómo la propagación de transacciones afecta llamadas entre servicios.

Evidencias que debes guardar

- [] Diagrama temporal de una transferencia exitosa y una fallida.
- [] Prueba automatizada de reversión completa.
- [] Nota de concurrencia con riesgo, estrategia elegida y escenario pendiente.

Preguntas de control

- 1 ¿Qué datos exactos deben compartir el mismo límite transaccional?
- 2 ¿Una excepción capturada y ocultada todavía provoca reversión? Investígalo en tu configuración.
- 3 ¿Qué ocurre si dos solicitudes leen el mismo saldo antes de guardar?
- 4 ¿La transacción resuelve automáticamente todos los problemas de concurrencia?
- 5 ¿Qué sacrificas al bloquear con mayor agresividad?

Errores comunes que debes vigilar

Suponer que varias operaciones de guardado son automáticamente indivisibles; colocar la transacción en la capa web; capturar el fallo antes de que el mecanismo pueda reaccionar; probar solo el camino feliz; confundir atomicidad con aislamiento.

Puerta de salida

Puedes provocar un fallo controlado y demostrar que no quedó ningún saldo ni movimiento parcial. También puedes explicar el riesgo concurrente aunque tu solución inicial sea sencilla.

Etapla 11. Construir una estrategia de pruebas

Una prueba es un experimento repetible. Las unitarias aíslan una pequeña decisión; las de integración verifican que varias piezas reales colaboren; las de capa web revisan el contrato HTTP. Ningún tipo reemplaza por completo a los demás.

Lo que debes crear

- [] Crear una matriz que relacione cada regla crítica con al menos una prueba.
- [] Probar servicios monetarios de forma aislada.
- [] Probar validación y traducción de errores en la capa web.
- [] Probar consultas y restricciones esenciales contra una base de pruebas.
- [] Crear un recorrido integrado de alta, billetera, depósito, transferencia e historial.
- [] Separar datos de prueba y limpiar el estado entre escenarios.

Antes de hacerlo, investiga

- Diferencia entre prueba unitaria, de integración y de extremo a extremo.
- Qué es un doble de prueba y qué significan simulación, sustituto y espía.
- Patrón preparar, actuar y comprobar.
- Qué aporta el contexto de Spring y por qué hace más lenta una prueba.
- Qué son pruebas de corte de una capa.
- Diferencia entre cobertura numérica y cobertura de riesgos.

Evidencias que debes guardar

- [] Matriz requisito-prueba.
- [] Informe de ejecución sin pruebas ignoradas injustificadamente.
- [] Una prueba que inicialmente falle por una razón correcta y luego pase.
- [] Descripción del alcance y limitación de cada tipo de prueba usado.

Preguntas de control

- 1 ¿Qué colaborador conviene sustituir al probar un servicio?
- 2 ¿Qué error solo descubrirías con una base real o equivalente?
- 3 ¿Qué hace frágil a una prueba aunque aumente la cobertura?
- 4 ¿Cómo sabes que una prueba puede fallar cuando la regla se rompe?
- 5 ¿Por qué compartir datos entre pruebas produce resultados intermitentes?

Errores comunes que debes vigilar

Arrancar toda la aplicación para cada detalle; verificar implementación en lugar de comportamiento; crear pruebas que siempre pasan; depender del orden; usar los mismos datos persistentes manuales; perseguir un porcentaje sin pensar en riesgos.

Puerta de salida

Las reglas críticas están conectadas a pruebas claras, el conjunto se ejecuta de forma repetible y puedes explicar qué fallo detectaría cada caso.

Etapas 12. Separar configuraciones

Una misma aplicación vive en contextos distintos. En pruebas necesita rapidez y aislamiento; en desarrollo necesita datos persistentes y diagnóstico; en producción necesitaría secretos externos y controles más estrictos. Un perfil agrupa diferencias sin duplicar el programa.

Lo que debes crear

- ☐ Definir una configuración común mínima.
- ☐ Definir un perfil local con PostgreSQL y valores externos.
- ☐ Definir un perfil de pruebas con base aislada.
- ☐ Crear valores de ejemplo seguros para orientar a otra persona.
- ☐ Excluir credenciales reales y resultados generados del repositorio.

Antes de hacerlo, investiga

- Qué es una propiedad y cómo se resuelve su prioridad.
- Qué es un perfil y cuándo se activa.
- Diferencia entre configuración, credencial y secreto.
- Qué significa externalizar configuración.
- Cómo una prueba obtiene su propia configuración.

Evidencias que debes guardar

- ☐ Tabla de propiedades por entorno sin valores secretos.
- ☐ Archivo de ejemplo que permita saber qué variables hacen falta.
- ☐ Comprobación del estado del repositorio antes de cada entrega.

Preguntas de control

- 1 ¿Qué cambia entre entorno y qué debe permanecer idéntico?
- 2 ¿Qué sucede si dos fuentes definen la misma propiedad?
- 3 ¿Por qué una contraseña de ejemplo puede seguir siendo peligrosa?
- 4 ¿Cómo evitarías que una prueba modifique tu base local?

Errores comunes que debes vigilar

Crear un perfil por capricho; repetir toda la configuración; confiar en ignorar un secreto después de haberlo subido; usar la base personal en pruebas; ocultar propiedades necesarias sin documentarlas.

Puerta de salida

Puedes ejecutar pruebas sin tocar datos locales y otra persona puede configurar el proyecto usando solo la documentación y valores propios.

Etapa 13. Documentar y preparar la entrega

La documentación es parte del producto. Debe permitir que una persona que no estuvo contigo entienda el problema, prepare el entorno, ejecute la aplicación, pruebe operaciones y conozca las decisiones importantes.

Lo que debes crear

- [] Escribir una portada breve del proyecto y su propósito.
- [] Documentar requisitos previos y pasos conceptuales de puesta en marcha.
- [] Describir configuración necesaria sin publicar secretos.
- [] Documentar operaciones, entradas, salidas y errores.
- [] Añadir diagramas de capas y datos.
- [] Registrar decisiones, limitaciones y mejoras futuras.
- [] Preparar una colección reproducible de solicitudes.

Antes de hacerlo, investiga

- Qué necesita saber una persona antes de clonar un proyecto.
- Diferencia entre documentación de uso y documentación de decisiones.
- Qué es una especificación de API.
- Cómo redactar ejemplos sin convertirlos en datos sensibles.
- Qué hace que un diagrama sea útil en vez de decorativo.

Evidencias que debes guardar

- [] Prueba de instalación realizada desde una carpeta limpia.
- [] Lista de problemas conocidos y su impacto.
- [] Colección ejecutada nuevamente contra la versión final.
- [] Guion de defensa oral de quince minutos.

Preguntas de control

- 1 ¿Puede otra persona ejecutar el proyecto sin preguntarte nada?
- 2 ¿La documentación describe lo que existe o lo que alguna vez planeaste?
- 3 ¿Qué decisión técnica es más importante defender?
- 4 ¿Qué limitación reconocerías antes de que el evaluador la descubra?

Errores comunes que debes vigilar

Escribir la documentación solo al final y olvidar pasos; asumir herramientas instaladas; publicar credenciales; documentar rutas antiguas; confundir cantidad de texto con claridad.

Puerta de salida

Una persona ajena puede preparar y probar el sistema, y tú puedes presentar problema, arquitectura, decisiones, demostración y mejoras dentro del tiempo previsto.

Etapa 14. Revisar como evaluador

Terminar no significa que una solicitud feliz haya funcionado una vez. La revisión final recorre requisitos, errores, pruebas, configuración y experiencia de uso. Debes intentar romper tu propia aplicación de manera controlada.

Lo que debes crear

- [] Recorrer uno por uno los escenarios A01 a A14.
- [] Ejecutar todas las pruebas desde un estado limpio.
- [] Buscar secretos, archivos generados y datos personales antes de publicar.
- [] Revisar nombres y responsabilidades de clases.
- [] Probar límites monetarios y entradas inesperadas.
- [] Comprobar persistencia, orden del historial y atomicidad.
- [] Preparar una versión identificable de la entrega.

Antes de hacerlo, investiga

- Qué es una lista de definición de terminado.
- Qué significa regresión.
- Cómo revisar dependencias no utilizadas o vulnerables.
- Diferencia entre refactorizar y cambiar comportamiento.
- Qué información aporta un historial de commits claro.

Evidencias que debes guardar

- [] Lista final firmada y fechada.
- [] Resultado de pruebas de la versión exacta entregada.
- [] Nota con tres fortalezas, tres riesgos y tres próximos pasos.

Preguntas de control

- 1 ¿Qué requisito tiene mayor riesgo financiero?
- 2 ¿Qué error sería más difícil de detectar manualmente?
- 3 ¿Qué parte cambiarías primero con una semana adicional?
- 4 ¿Qué decisión puedes defender con evidencia y cuál fue solo una suposición?

Errores comunes que debes vigilar

Agregar una función nueva durante la revisión; cambiar varias piezas sin volver a probar; ocultar limitaciones; limpiar el historial destruyendo contexto útil; entregar una versión distinta de la verificada.

Puerta de salida

Los catorce escenarios pasan, las pruebas están verdes, no hay secretos y la versión entregada coincide con la revisada.

PARTE III

Tarjetas de investigación obligatoria

19. Cómo completar una tarjeta

Cada tarjeta cubre una pieza que deberás comprender antes de usarla. No la completes copiando una definición. Produce una explicación propia, una analogía, un lugar concreto de tu proyecto donde aparece, un error que previene y una fuente consultada.

Evidencia por tarjeta	Criterio mínimo
Definición propia	Entre dos y cinco frases sin copiar la fuente.
Analogía	Debe conservar la relación importante, no solo sonar simpática.
Aplicación	Señala una clase, configuración, prueba o decisión de tu proyecto.
Riesgo	Describe qué podría salir mal si interpretas el concepto de forma incorrecta.
Fuente	Prioriza documentación oficial y registra título y enlace.
Pregunta pendiente	Si todavía queda una duda, escríbela con precisión.

Regla de honestidad

Marca una tarjeta como comprendida solo si puedes explicarla sin mirar y responder una repregunta. Reconocer una palabra no equivale a dominar el concepto.

A. Java y construcción

ID	Tema	Misión de investigación
R01	JDK, JRE y JVM	Explica qué aporta cada uno al desarrollo y a la ejecución.
R02	Compilación y bytecode	Sigue el recorrido desde un archivo fuente hasta la ejecución.
R03	Classpath	Investiga cómo se encuentran clases propias y externas.
R04	Maven	Distingue descripción del proyecto, resolución de dependencias y ciclo de construcción.
R05	Coordenadas	Explica grupo, artefacto y versión como identidad de una biblioteca.
R06	Alcance de dependencia	Compara dependencias necesarias al compilar, ejecutar y probar.
R07	Plugin de construcción	Diferencia una herramienta que actúa durante la construcción de una biblioteca usada por la aplicación.
R08	Wrapper	Explica por qué el proyecto incluye su forma preferida de ejecutar Maven.

B. Núcleo de Spring

ID	Tema	Misión de investigación
R09	Contenedor	Explica qué objetos administra y qué problema evita.
R10	Bean	Distingue un objeto administrado de uno creado libremente.
R11	Inversión de control	Describe quién toma la iniciativa de crear y conectar objetos.
R12	Inyección de dependencias	Compara recibir una colaboración con construirla internamente.
R13	Inyección por constructor	Investiga sus efectos en obligatoriedad, inmutabilidad y pruebas.
R14	Descubrimiento de componentes	Averigua desde dónde busca Spring y qué ocurre fuera de ese alcance.
R15	Configuración automática	Explica qué señales utiliza Spring Boot para tomar decisiones iniciales.
R16	Ciclo de vida	Investiga cuándo nace, se prepara y se destruye un objeto administrado.

C. Web y contratos

ID	Tema	Misión de investigación
R17	Solicitud HTTP	Identifica método, dirección, cabeceras, parámetros y cuerpo.
R18	Respuesta HTTP	Identifica estado, cabeceras y cuerpo.
R19	Semántica de métodos	Compara lectura, creación, reemplazo, modificación parcial y eliminación.
R20	Estados HTTP	Construye un mapa de éxitos, errores del cliente y errores del servidor.
R21	Recurso	Explica por qué una API modela sustantivos aunque ejecute acciones.
R22	Idempotencia	Analiza qué ocurre cuando la red repite una solicitud monetaria.
R23	Serialización	Describe cómo un objeto se convierte en JSON y vuelve a convertirse.
R24	DTO	Justifica la separación entre contrato externo y modelo persistente.
R25	Validación	Distingue forma de datos, existencia y regla de negocio.
R26	Paginación	Compara página por desplazamiento y cursor para historiales que crecen.

D. Persistencia

ID	Tema	Misión de investigación
R27	Base relacional	Explica tablas, filas, columnas, claves y relaciones.
R28	ORM	Describe qué traducción automatiza y qué no puede decidir por ti.
R29	JPA	Diferencia especificación, proveedor y herramientas de Spring Data.
R30	Entidad	Explica identidad, estado persistente y ciclo de vida.
R31	Clave primaria	Compara identificador interno con documento de negocio.
R32	Clave foránea	Explica cómo protege relaciones válidas.
R33	Restricción única	Investiga por qué complementa una comprobación previa.
R34	Cardinalidad	Representa uno a uno, uno a muchos y muchos a muchos.
R35	Carga diferida	Analiza beneficio, sorpresa frecuente y límite de sesión.
R36	Problema N+1	Explica cómo una consulta inicial puede multiplicarse sin notarlo.
R37	Migración	Distingue evolución controlada del esquema y generación automática.
R38	Consulta derivada	Investiga cuándo un nombre expresa bien una consulta y cuándo deja de hacerlo.

E. Dinero, tiempo y consistencia

ID	Tema	Misión de investigación
R39	Decimal exacto	Explica por qué el dinero no debe depender de aproximaciones binarias.
R40	Escala y redondeo	Distingue cantidad de decimales, precisión y política de redondeo.
R41	Tiempo	Compara instante, fecha y hora local y zona horaria.
R42	Transacción	Describe qué agrupa y cuándo confirma o revierte.
R43	ACID	Relaciona cada propiedad con una transferencia concreta.
R44	Aislamiento	Investiga qué puede observar una operación mientras otra trabaja.
R45	Actualización perdida	Explica cómo dos retiros válidos por separado pueden producir un saldo incorrecto.
R46	Bloqueo optimista	Investiga detección de conflictos y necesidad de reintento.
R47	Bloqueo pesimista	Investiga exclusión, espera y riesgo de bloqueo mutuo.
R48	Auditoría	Explica por qué saldo resultante e identificador común ayudan a reconstruir hechos.

F. Pruebas y operación

ID	Tema	Misión de investigación
R49	Prueba unitaria	Define su unidad, aislamiento y velocidad esperada.
R50	Prueba de integración	Identifica fronteras reales que conviene verificar juntas.
R51	Doble de prueba	Distingue sustitución útil de simulación excesiva.
R52	Pirámide de pruebas	Relaciona cantidad, velocidad, realismo y costo de diagnóstico.
R53	Perfil de pruebas	Explica cómo evita usar configuración y datos locales.
R54	Registro	Distingue información diagnóstica de datos sensibles.
R55	Variable de entorno	Explica cómo externaliza configuración sin resolver por sí sola la seguridad.
R56	Comprobación de salud	Investiga qué indica que una aplicación está viva y preparada.

20. Plantilla para una tarjeta

Campo	Completar
ID y concepto	
Definición con mis palabras	
Analogía propia	
Dónde aparece en Billetera Clara	
Qué error ayuda a prevenir	
Fuente oficial consultada	
Segunda fuente o experimento	
Pregunta que todavía tengo	
Explicado oralmente sin mirar	☐ Sí ☐ Todavía no

Escalera de pistas

Si te bloqueas: 1) vuelve a formular la pregunta; 2) busca el concepto en la documentación oficial; 3) busca un ejemplo mínimo distinto de tu proyecto; 4) realiza un experimento pequeño; 5) pide una pista conceptual; 6) solo al final compara implementaciones y explica cada diferencia.

PARTE IV

Plan de ejecución y seguimiento

21. Plan sugerido de 10 sesiones

Sesión	Foco	Trabajo principal	Tiempo
1	Entorno y proyecto base	Etapas 0 a 2; tarjetas R01 a R16	3 h
2	Análisis y arquitectura	Etapa 3, mapas y casos de uso	3 h
3	Dominio	Etapa 4; tarjetas R39 a R41 y R48	3 h
4	Persistencia	Etapa 5; tarjetas R27 a R38	4 h
5	Contratos	Etapa 6; tarjetas R17 a R26	3 h
6	Casos de uso	Etapa 7; depósito y retiro primero	4 h
7	Web y errores	Etapas 8 y 9	3 h
8	Transferencia	Etapa 10; tarjetas R42 a R47	4 h
9	Pruebas y configuración	Etapas 11 y 12; tarjetas R49 a R56	3 h
10	Documentación y defensa	Etapas 13 y 14	2 h

Los tiempos son una referencia, no una obligación. Si una puerta de salida no se cumple, continúa en la misma etapa. La meta no es respetar un cronómetro: es evitar avanzar con huecos conceptuales que luego convierten cada error en misterio.

22. Registro de sesión

Pregunta	Registro
Fecha y duración real	
Objetivo concreto de hoy	
Conceptos investigados	
Cambio observable producido	
Error más útil encontrado	
Hipótesis probadas	
Evidencia de que funciona	
Qué podría romperse todavía	
Primer paso de la próxima sesión	

23. Lista de control por hitos

Hito 1: aplicación mínima

- ☐ El proyecto construye sin modificaciones accidentales.
- ☐ La aplicación arranca y se detiene correctamente.
- ☐ Comprendo la estructura generada y no edito resultados de construcción.
- ☐ El repositorio no contiene secretos ni archivos generados.

Hito 2: dominio y datos

- ☐ El diagrama representa todos los conceptos obligatorios.
- ☐ Dinero, tiempo e identificadores están justificados.
- ☐ Las restricciones importantes existen también donde corresponde en la base.
- ☐ Puedo guardar y recuperar sin depender de la capa web.

Hito 3: operaciones

- ☐ Depósito, retiro y transferencia respetan todas las precondiciones.
- ☐ Cada éxito genera el historial exacto esperado.
- ☐ Cada rechazo conserva el estado anterior.
- ☐ La transferencia es indivisible.

Hito 4: contrato web

- ☐ Métodos, rutas y estados son coherentes.
- ☐ Entradas y salidas no dependen directamente de la persistencia.
- ☐ Validaciones y errores comparten un formato estable.
- ☐ Los catorce escenarios pueden reproducirse externamente.

Hito 5: entrega

- ☐ Las pruebas cubren riesgos, no solo líneas.
- ☐ Los perfiles aíslan prueba y desarrollo.
- ☐ La documentación permite ejecutar desde cero.
- ☐ Puedo defender decisiones y reconocer limitaciones.

24. Semáforo de comprensión

Color	Qué significa	Qué hacer
Verde	Puedo explicarlo, aplicarlo y detectar un uso incorrecto.	Avanzar y revisar nuevamente al final.
Amarillo	Lo reconozco y puedo usarlo siguiendo una guía, pero no responder repreguntas.	Crear un experimento pequeño y completar la tarjeta.
Rojo	No sé qué problema resuelve o dependo de copiar una solución.	Detener la etapa, reformular la duda y buscar fuentes básicas.

PARTE V

Defensa e entrevista técnica

25. Guion de defensa de 15 minutos

Minutos	Tema	Qué debes demostrar
0 a 2	Problema	Qué resuelve la API, quién la usaría y cuáles son las reglas de mayor riesgo.
2 a 5	Arquitectura	Recorrido de una petición y fronteras entre capas.
5 a 8	Datos	Modelo, relaciones, dinero, historial y decisiones de tiempo.
8 a 11	Demostración	Un caso exitoso y un rechazo que conserva el estado.
11 a 13	Pruebas	Qué riesgos están protegidos y cómo demuestras atomicidad.
13 a 15	Aprendizaje	Decisión difícil, limitación conocida y siguiente mejora prioritaria.

Una defensa convincente

No consiste en nombrar muchas tecnologías. Consiste en enlazar causa y efecto: elegí esto porque protege esta regla; lo comprobé con esta prueba; esta alternativa tendría este costo.

26. Preguntas que podría hacer el evaluador

- 1 ¿Qué hace Spring Boot desde que ejecutas la aplicación hasta que queda escuchando solicitudes?
- 2 ¿Qué diferencia hay entre el contenedor de Spring y una colección normal de objetos Java?
- 3 ¿Por qué recibes las dependencias en vez de construirlas dentro de cada clase?
- 4 ¿Qué diferencia hay entre controlador, servicio y repositorio?
- 5 ¿Dónde ubicastes la regla de fondos suficientes y por qué?
- 6 ¿Qué parte del proyecto conoce HTTP y qué parte podría reutilizarse fuera de la web?
- 7 ¿Por qué separaste los objetos de entrada y salida de las entidades?
- 8 ¿Cómo se transforma una solicitud JSON en datos utilizables por el servicio?
- 9 ¿Qué diferencia existe entre una validación de formato y una regla de negocio?
- 10 ¿Cómo elegiste los estados HTTP para creación, ausencia, dato inválido y conflicto?
- 11 ¿Por qué no usaste un número de punto flotante binario para dinero?
- 12 ¿Qué política aplicas a escala y redondeo?
- 13 ¿Cómo representas fecha y hora y qué ocurre con zonas horarias diferentes?
- 14 ¿Por qué el documento del cliente no necesariamente debe ser la clave primaria?
- 15 ¿Qué restricciones protege la base aunque exista validación en Java?
- 16 ¿Qué diferencia hay entre JPA, el proveedor ORM y Spring Data?
- 17 ¿Qué peligro tienen las relaciones cargadas automáticamente?
- 18 ¿Cómo se ordena el historial y dónde se realiza ese orden?

- 19 ¿Por qué un movimiento no se puede editar mediante la API común?
- 20 ¿Cómo relacionas los dos lados de una transferencia?
- 21 ¿Qué significa que la transferencia sea atómica?
- 22 ¿Cómo probaste que un fallo intermedio revierte todo?
- 23 ¿Qué ocurriría con dos retiros simultáneos sobre el mismo saldo?
- 24 ¿Qué diferencia existe entre atomicidad y aislamiento?
- 25 ¿Qué estrategia de concurrencia elegiste y qué costo tiene?
- 26 ¿Qué prueba consideras más valiosa y qué regresión detecta?
- 27 ¿Cuándo usas una prueba unitaria y cuándo una de integración?
- 28 ¿Por qué no arrancar siempre toda la aplicación para cada prueba?
- 29 ¿Cómo evitas que las pruebas modifiquen tu base de desarrollo?
- 30 ¿Qué información pública contiene un error y qué conservas solo en registros?
- 31 ¿Cómo agregarías idempotencia a los depósitos y transferencias?
- 32 ¿Qué cambiaría si un cliente pudiera tener billeteras en varias monedas?
- 33 ¿Qué migración necesitarías si el modelo cambia después de tener datos reales?
- 34 ¿Qué decisión de tu diseño te genera más dudas hoy?
- 35 Si tuvieras una semana más, ¿qué mejorarías primero y por qué?

27. Hoja de autoevaluación

Escala: 0 = no lo reconozco; 1 = lo reconozco; 2 = lo hago con ayuda; 3 = lo explico y aplico; 4 = puedo detectar errores y enseñar la idea.

Afirmación	Puntaje
Puedo explicar el arranque y la configuración automática sin llamarlos magia.	0 1 2 3 4
Puedo seguir una petición por todas las capas.	0 1 2 3 4
Puedo justificar dinero, tiempo e identificadores.	0 1 2 3 4
Puedo explicar la diferencia entre validación, negocio y persistencia.	0 1 2 3 4
Puedo demostrar que una transferencia no queda a medias.	0 1 2 3 4
Puedo distinguir y elegir tipos de pruebas.	0 1 2 3 4
Puedo interpretar un error de arranque usando los registros.	0 1 2 3 4
Puedo defender mis rutas, estados y contratos.	0 1 2 3 4
Puedo reconocer riesgos de concurrencia y seguridad pendientes.	0 1 2 3 4
Puedo ejecutar y presentar el proyecto desde un entorno limpio.	0 1 2 3 4

Resultado total: ____ / 40

- 0 a 14: vuelve al mapa mental y completa las tarjetas fundamentales.
- 15 a 24: ya puedes construir con guía, pero necesitas consolidar explicaciones y errores.
- 25 a 32: nivel sólido para defender un primer proyecto junior.
- 33 a 40: buena comprensión; busca ahora profundidad en concurrencia, seguridad y operación.

PARTE VI

Cierre y definición de terminado

28. Lista final de requisitos

- | | |
|---|---|
| ☐ Clientes: alta, consulta, unicidad y validación. | ☐ Errores: formato uniforme, detalles de campo y seguridad. |
| ☐ Billeteras: alta única por cliente, moneda, estado y saldo inicial protegido. | ☐ Persistencia: relaciones, restricciones y perfil persistente. |
| ☐ Depósitos: monto válido, actualización exacta y movimiento. | ☐ Pruebas: reglas críticas, reversión y recorrido integrado. |
| ☐ Retiros: fondos suficientes, actualización exacta y movimiento. | ☐ Configuración: perfiles, valores externos y ausencia de secretos. |
| ☐ Transferencias: origen y destino distintos, dos movimientos relacionados y atomicidad. | ☐ Documentación: ejecución, API, diagramas, decisiones y limitaciones. |
| ☐ Historial: orden, saldo resultante, filtros y ausencia de edición ordinaria. | ☐ Git: historial comprensible y versión final identificable. |
| ☐ API: recursos coherentes, estados correctos y contratos separados. | ☐ Defensa: demostración reproducible y respuestas con causa y efecto. |

29. Declaración de autoría y comprensión

Completa esta declaración cuando termines. Su propósito no es prohibir herramientas: es asegurarte de que las herramientas no hayan reemplazado tu aprendizaje.

Declaración

Puedo explicar las partes principales de esta solución con mis propias palabras. Cuando utilicé una sugerencia externa, la comprobé, la adapté y registré lo que comprendí. Sé reproducir los escenarios críticos, identificar las limitaciones actuales y explicar qué investigaría para continuar.

Nombre: _____

Fecha: _____

Versión o referencia de la entrega: _____

Firma personal: _____

30. Última reflexión

Cuando termines, no midas el progreso solo por la cantidad de clases creadas. Mídelo por preguntas que ahora puedes responder: qué ocurre al arrancar, por qué una solicitud llega a cierta capa, cómo se protege una regla, qué garantiza una transacción y qué evidencia ofrece una prueba. Ese mapa mental es transferible a cualquier proyecto Spring Boot que construyas después.

Tu siguiente paso

No necesitas saber Spring Boot antes de empezar. Convierte cada parte desconocida en una pregunta concreta. Crea tu diario de decisiones, completa la Etapa 0 y no agregues todavía clases de negocio.